

Lab Assignment 4

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:

Mridul Sharma

Roll No. 1024150204

Batch: 2023

1) Create a class named 'Rectangle' with two data members – *length* and *breadth* and a function to calculate the area which is *length*breadth*. The class has three constructors which are:

(a) having no parameter – values of both length and breadth are assigned zero.

(b) having two numbers as parameters – the two numbers are assigned as length and breadth respectively.

(c) having one number as parameter – both length and breadth are assigned that number.

Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

Input: -

WRITTEN CODE

```
#include <iostream>

using namespace std;

class Rectangle{

    public:

    int length, breadth;

    Rectangle(){};

    Rectangle(int a, int b){

        length = a; breadth = b;

    }

    Rectangle(int x){

        length = x; breadth = x;

    }

    void area(){

        int area = length*breadth;

        cout << area;

    };

};

int main(){
```

```

Rectangle A2(79,7);

A2.area();

cout << " ";

Rectangle A1(67);

A1.area();

}

```

```

a4q1.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Rectangle{
4      public:
5      int length, breadth;
6      Rectangle(){};
7      Rectangle(int a, int b){
8          length = a; breadth = b;
9      }
10     Rectangle(int x){
11         length = x; breadth = x;
12     }
13     void area(){
14         int area = length*breadth;
15         cout << area;
16     };
17 };
18 int main(){
19     Rectangle A2(79,7);
20     A2.area();
21     cout << " ";
22     Rectangle A1(67);
23     A1.area();
24 }

```

Output: -

```

PS C:\Users\Harshita\Desktop\OOPS\Assignments\A4> cd "c:\Users\Harshita\Desktop\OOPS\Assignments\A4\" ; if ($?) { g
+ a4q1.cpp -o a4q1 } ; if ($?) { .\a4q1 }
553 4489
PS C:\Users\Harshita\Desktop\OOPS\Assignments\A4>

```

2) Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

WRITTEN CODE

```
#include <iostream>

using namespace std;

class Rectangle {
    int length, breadth;
public:
    Rectangle() {
        length = 0; breadth = 0;
        cout << "Default constructor called\n";
    }
    Rectangle(int l, int b) {
        length = l;
        breadth = b;
        cout << "Parameterized constructor called\n";
    }
    ~Rectangle() {
        cout << "Destructor called\n";
    }
    int area() {
        return length * breadth;
    }
};

int main() {
    Rectangle arr[3] = {
        Rectangle(),
        Rectangle(5, 4),
```

```

    Rectangle(7, 3)

};

for (int i = 0; i < 3; i++) {

    cout << "Area of object " << i+1 << ": " << arr[i].area() << endl;

}

return 0;}

```

```

1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      int length, breadth;
5  public:
6      Rectangle() {
7          length = 0; breadth = 0;
8          cout << "Default constructor called\n";
9      }
10     Rectangle(int l, int b) {
11         length = l;
12         breadth = b;
13         cout << "Parameterized constructor called\n";
14     }
15     ~Rectangle() {
16         cout << "Destructor called\n";
17     }
18     int area() {
19         return length * breadth;
20     }
21 };
22 int main() {
23     Rectangle arr[3] = {
24         Rectangle(),
25         Rectangle(5, 4),
26         Rectangle(7, 3)
27     };
28     for (int i = 0; i < 3; i++) {
29         cout << "Area of object " << i+1 << ": " << arr[i].area() << endl;
30     }
31     return 0;}

```

```
Output Clear
Default constructor called
Parameterized constructor called
Parameterized constructor called
Area of object 1: 0
Area of object 2: 20
Area of object 3: 21
Destructor called
Destructor called
Destructor called

=== Code Execution Successful ===
```

3) Verify the following about destructor by writing the program:

- (i) Name should begin with tilde sign (~) and must match class name.**
- (ii) There cannot be more than one destructor in a class.**
- (iii) Destructors do not allow any parameter.**
- (iv) They do not have any return type, just like constructors.**

When you do not specify any destructor in a class, compiler generates a default destructor and inserts it into your code.

Answers :-

- i) True
- ii) True
- iii) True
- iv) True

```

1  #include <iostream>
2  using namespace std;
3  class Rectangle{
4      public:
5          int length, breadth;
6
7          Rectangle(){
8              cout<< "Constructor is called"<< endl;
9          };
10         ~Rectangle(){
11             cout<< "Destructor is called"<< endl;
12         };
13     };
14
15     int main(){
16         Rectangle();
17     }
18

```

```

PS C:\Users\Harshita\Desktop\OOPS\Assignments\A4> cd "c:\Users\Harshita\Desktop\OOPS\Assignments\A4" & gcc -o des des.cpp -o des ; if ($?) { .\des }
Constructor is called
Destructor is called
PS C:\Users\Harshita\Desktop\OOPS\Assignments\A4>

```

```

1  #include <iostream>
2  using namespace std;
3
4  class Demo {
5      public:
6          Demo() {
7              cout << "Constructor called\n";
8          }
9
10         ~Demo() {
11             cout << "Destructor called\n";
12         }
13     };
14
15     int main() {
16         Demo obj;
17         return 0;
18     }

```

Output

```
Constructor called  
Destructor called
```

```
=== Code Execution Successful ===
```

4) Implement dynamic memory allocation. Use new and delete keywords. (For integer variable, float variable, integer array, float array, class objects, Array of Objects)

WRITTEN CODE

```
#include <iostream>

using namespace std;

class Test {
public:
    int x;

    Test(int val) {
        x = val;
    }

    void show() {
        cout << "Value: " << x << endl;
    }
};

int main() {
    // Integer
    int *p = new int;
    *p = 10;
    cout << "Integer: " << *p << endl;
    delete p;
```



```
// Float

float *f = new float;

*f = 5.5;

cout << "Float: " << *f << endl;

delete f;

// Integer array

int *arr = new int[3];

for(int i = 0; i < 3; i++) {

    arr[i] = i + 1;

    cout << arr[i] << " ";

}

cout << endl;

delete[] arr;

// Object

Test *obj = new Test(100);

obj->show();

delete obj;

// Array of objects

Test *objArr = new Test[2]{Test(1), Test(2)};

for(int i = 0; i < 2; i++) {

    objArr[i].show();

}

delete[] objArr;

return 0;

}
```

```

1  #include <iostream>
2  using namespace std;
3  class Test {
4  public:
5      int x;
6      Test(int val) {
7          x = val;
8      }
9      void show() {
10         cout << "Value: " << x << endl;
11     }
12 };
13 int main() {
14     // Integer
15     int *p = new int;
16     *p = 10;
17     cout << "Integer: " << *p << endl;
18     delete p;
19     // Float
20     float *f = new float;
21     *f = 5.5;
22     cout << "Float: " << *f << endl;
23     delete f;
24     // Integer array
25     int *arr = new int[3];
26     for(int i = 0; i < 3; i++) {
27         arr[i] = i + 1;
28         cout << arr[i] << " ";
29     }
30     cout << endl;
31     delete[] arr;

```

```

32     // Object
33     Test *obj = new Test(100);
34     obj->show();
35     delete obj;
36     // Array of objects
37     Test *objArr = new Test[2]{Test(1), Test(2)};
38     for(int i = 0; i < 2; i++) {
39         objArr[i].show();
40     }
41     delete[] objArr;
42     return 0;
43 }

```

Output

Clear

Integer: 10

Float: 5.5

1 2 3

Value: 100

Value: 1

Value: 2

=== Code Execution Successful ===